

App Dev Project Report

1. Student Details

Name: Ujjawal Bhardwaj

Roll Number: 24f1000062

Email: 24f1000062@ds.study.iitm.ac.in

About Me:

I am a student in the **IIT Madras BS Degree Program in Data Science** with a strong interest in **software development, web applications, and data-driven systems**. I enjoy building practical applications that solve real world problems and improve user experiences. My focus is on developing scalable systems using modern technologies such as Flask, databases, and data analysis tools.

2. Project Details

Project Title: Placement Portal(Naukri Shala)

Problem Statement:

The traditional placement process in many institutions involves manual coordination between students, companies, and administrators. This often leads to inefficiencies such as delayed communication, difficulty in tracking applications, and lack of centralized information.

The objective of this project is to design and develop a **web based placement management system** that allows students to apply for job opportunities, companies to post job drives, and administrators to manage the entire recruitment process efficiently.

Approach:

The application was developed using the **Flask web framework** with a modular architecture. The system includes three primary roles:

- **Students** who can register, view job drives, and apply for positions.
- **Companies** who can register, create recruitment drives, and review student applications.
- **Administrators** who manage approvals, monitor companies, and oversee the placement process.

The backend is implemented using **Flask and SQLAlchemy**, while the frontend uses **HTML, Bootstrap, and Jinja2 templates** for dynamic rendering.

3. AI / LLM Declaration

I used **ChatGPT (GPT-5)** to assist with:

- Improving the structure of Flask routes and modular project architecture
- Debugging SQLAlchemy database errors
- Writing parts of documentation and improving template structure

The approximate usage of AI assistance is **10–15%**, mainly for guidance and debugging suggestions.

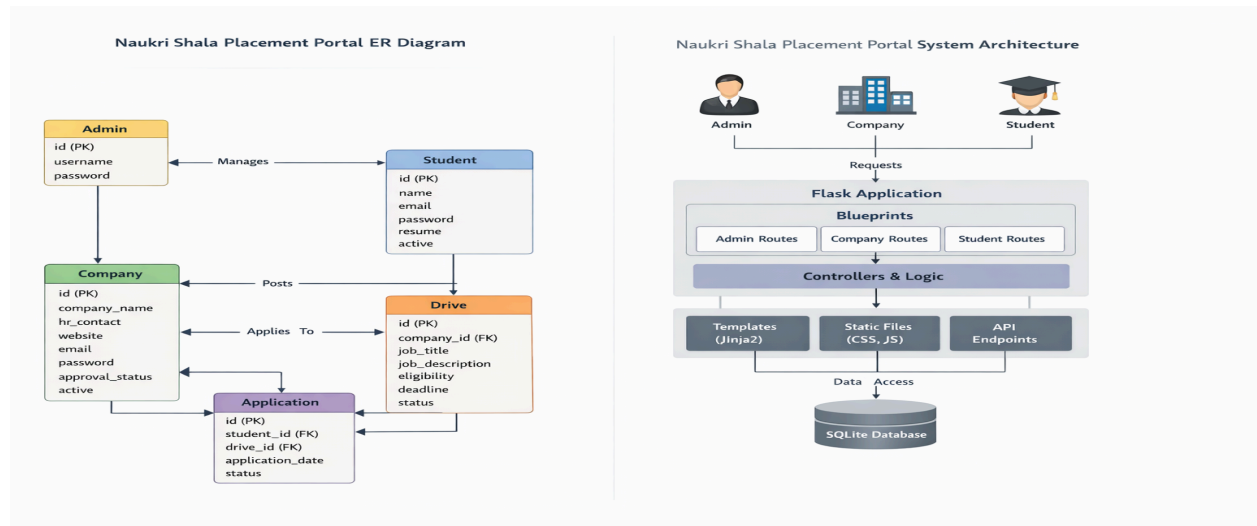
All major components including **project logic, debugging, database design, and final integration** were implemented and tested manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	ORM for managing database tables and queries
Jinja2	Template engine for rendering dynamic HTML
Bootstrap 5	Responsive UI and frontend styling
SQLite	Lightweight database for storing application data
HTML / CSS	Structure and styling of web pages
Flask Sessions	User authentication and session management

5. Database Schema / ER Diagram

The application uses a relational database with the following tables:



Tables

Admin

- id
- username
- password

Student

- id
- name
- email
- password
- resume
- active

Company

- id
- company_name
- hr_contact
- website
- email
- password
- approval_status
- active

Drive

- id

- company_id
- job_title
- job_description
- eligibility
- deadline
- status

Application

- id
- student_id
- drive_id
- application_date
- status

Relationships

- **One-to-Many** → **Company** → **Drive**
- **One-to-Many** → **Student** → **Application**
- **One-to-Many** → **Drive** → **Application**

(Insert **ER Diagram Image** here created using **dbdiagram.io** / **draw.io**)

6. API Resource Endpoints

Endpoint	Method	Description
/login	POST	Authenticate user (student/company/admin)
/student/dashboard	GET	Display student dashboard
/company/dashboard	GET	Display company dashboard
/company/create_drive	POST	Create a new job drive
/student/apply/<drive_id>	POST	Student applies to a drive
/company/applications/<drive_id>	GET	View applicants for a job drive
/admin/dashboard	GET	Admin management dashboard

YAML API Definition File:

A simple OpenAPI YAML file describing the application routes is included in the submission ZIP as api.yaml.

7. Architecture and Features (Optional)

Architecture Overview

app.py
config.py
models.py
routes/
 auth.py
 admin.py
 student.py
 company.py
templates/
static/

Description

app.py

- Main entry point of the application
- Initializes Flask and registers Blueprints

models.py

- Contains SQLAlchemy database models

routes/

- Contains Flask Blueprints for modular routing
- Separate route files for admin, student, company, and authentication

templates/

- Jinja2 HTML templates for rendering pages

static/

- CSS and frontend assets
-

Implemented Features

- ✓ Student registration and login
 - ✓ Company registration and admin approval system
 - ✓ Job drive creation by companies
 - ✓ Student job applications
 - ✓ Company dashboard for managing applicants
 - ✓ Admin dashboard for system monitoring
 - ✓ Session-based authentication system
 - ✓ Dynamic HTML rendering using Jinja2
-

Additional Features

- Application status tracking (Pending / Accepted / Rejected)
 - Company approval workflow by admin
 - Dashboard showing number of applicants per drive
-

8. Video Presentation

Drive Link:

 **Recording 2026-03-10 183620.mp4**
(Accessible to anyone with **View** permission)